A Maude Framework for Efficient Analysis of Multiformalism Models

Lorenzo Capra[1] [0000−0002−1029−1169]

Università degli Studi di Milano
Via Celoria 18, Milan, Italy
capra@di.unimi.it

Abstract. We propose a new quantitative analysis method for realistic adaptive systems that unifies two strands of work: (1) a compositional, symmetry-aware construction of a quotient transition system for lumped Markov processes in Rewritable (Stochastic) Petri nets (RwSPNs) using algebraic operators and structured node labeling, and (2) the use of Maude as a multiformalism modeling environment in which multiple formalisms coexist within one framework. We generalize the compositional RwSPN approach to this multiformalism setting and present encouraging preliminary results from an edge-computing case study.

This work is based on a recently proposed approach to precisely compute Markov chain transition rates from Maude's labeled transition systems.

Keywords: Maude · Multiformalism models · Reconfigurable distributed systems.

1 Introduction

Maude [15] is a purely declarative, high-performance, and expressive language with rewriting logic semantics [6]. A Maude system module is an executable specification of a distributed system. The Maude runtime support has various built-in model checking facilities. Maude has also been used as a logical framework for other formalisms, such as Petri Nets (PN), which lack native support for intuitively modeling adaptable systems.

In this paper, we present a quantitative analysis methodology for realistic adaptive systems that unifies two existing approaches. The first derives quotient transition systems for lumped Markov processes in Rewritable (Stochastic) Petri nets (RwSPNs) via compositional algebraic operators and structured node labeling, capturing symmetries as graph morphisms [9,13,11]. The second uses Maude as a multiformalism environment to represent complex systems through multiple coexisting formalisms in a unified framework [3]. We extend the modular RwSPN approach to this multiformalism setting by modeling interconnected nodes in a heterogeneous network as instances of generic "monoid" structures with symmetric compositional operators, generalizing those in [13].

The method can be seamlessly integrated with a recent approach [10] that precisely determines the state-transition rates of the continuous-time Markov chain (CTMC) induced by the labeled transition system (TS) generated from an executable specification in Maude with stochastic parameters. Exploiting symmetries yields a TS quotient whose strong bisimulation guaranties the *exact lumpability* of the corresponding CTMC.

Related Works Several approaches to timed and probabilistic analysis are available in Maude. [24] provides a survey, although it is no longer current. The framework proposed in [33] supports deterministic time specifications for the analysis of real-time systems. The method introduced in [1], which relies on probabilistic rewrite theories associated with actors, supports probabilistic discrete-event simulation. More recently, [29] proposed an extensive approach for applying Maude to stochastic analysis through a probabilistic extension of its strategy language. The rewriting-logic semantics associates a labeled transition system with ground terms; each state transition may involve many rewrites. The method in [10] builds a Markov process directly from Maude specifications with stochastic parameters, using object-level "meta-operators" to enumerate rule matches and compute transition rates. Extending this to a multiformalism framework improves the modeling of realistic systems. The use of behavioral equivalences within the Maude framework is well established (see, e.g., [22]). Our method, however, differs in that it focuses on identifying symmetries that mirror the modular structure of models constructed and manipulated via compositional operators.

The paper first reviews the compositional method of [13,11] for constructing a lumped CTMC from rewritable (stochastic) PNs. It then extends this technique to a multiformalism framework whose interconnected nodes are monoid structures sharing a distributed state. Preliminary results are presented on an edge-computing case study; a conclusion outlines contributions and future work. For the sake of completeness, several code snippets are included throughout the work. It is assumed that the reader already has knowledge of Maude syntax and the basic notions of rewriting logic. An intuitive presentation of the PT formalism [28] is provided in Section 2.

2 Modular rewritable (stochastic) Petri nets

In this section, we summarize the approach introduced in [13,11] and extend the underlying PN class to fit the requirements of the case study (Section 4). We extensively use *parameterized* modules, in which *theories* specify interfaces via syntactic and semantic constraints on parameter modules. Views (which can be parameterized in turn) link theories to the target modules by mapping their sorts and operators.

A rewritable Place-Transition net (RwPT) [27,26,13] is an algebraic formalism for mutable systems that combines the firing rule with structural rewriting. We consider its stochastic variant (RwSPN), introduced in [11] and extended in [10], where PT nets are enriched with bounded places and flush arcs; the encoding in Maude is available in a small module hierarchy in https://github.com/lgcapra/rewpt.

PT nodes are defined by the modules `PLACE{PL:: TRIV}` and `TRAN{L:: TRIV}`, whose parameters are node labels (the theory `TRIV` only declares the formal sort `Elt`); this flexible labeling is central. Multisets of places (module `PBAG{PL:: TRIV}`) are weighted sums: for instance, `3 . p(1) + 1 . p(2)` of sort `Pbag` (from `PBAG{Nat}`) denotes three copies of p_1 and one of p_2. The associative-commutative sum `_+_` (identity `nilP`) is a constructor. The places may be bounded; e.g., `p("a", 3)` (from `PLACE{String}`) has capacity 3, while `p("a")` is unbounded. The following excerpts introduce the PT signature, parameterized by node labels. This construction is based on the predefined `MAP` module via the (parameterized) `Ematrix` view.

```
fmod PT-NET{L :: TRIV, PL :: TRIV} is
 generated-by MAP{L, Ematrix{PL}} * (sort Map{L, Ematrix{PL}} to Net,
    sort Entry{L, Ematrix{PL}} to Tran, op emptyM to emptyNet) .
 var L : L$Elt . var T : Tran . vars N N' : Net . var P : Place .
 var M : Pbag . var K : NzNat . var EQ : Ematrix . var Q : Tmatrix . var S : Pset . var F :
      FlushArcs .
 op l : Tran -> L$Elt . op q : Tran -> Ematrix .
 op m : Tran -> Tmatrix . op f : Tran -> FlushArcs .
 ops I O H : Tran -> Pbag .
 eq l(L |-> EQ) = L .
 eq I(T) = I(m(T)) .
 ...
 eq L |-> [nilP, nilP] = emptyNet . *** removes isolated transitions
 op enabled : Tran Pbag -> [Bool] . *** doesn't consider place capacity
 eq enabled(L |-> EQ, M) = enabled(EQ, M) .
 op firing : Tran Pbag -> Pbag .
 eq firing(L |-> EQ, M) = firing(EQ, M) .
endfm

fmod PT-SYS{L :: TRIV, PL :: TRIV} is
  protecting PT-NET{L, PL} .
  sort System .
  var N : Net . var M : Pbag .
  op __ : Net Pbag -> System [ctor] .
  op n : System -> Net . op m : System -> Pbag .
  ...
endfm
```

Transitions (terms of sort `Tran`) are designated by labels linked to local adjacency lists, which by default are `Pbag` triples in `[]` (terms of sort `Tmatrix`). A typical transition is `t(L) |-> [I,O,H]` where `L` is the label and `I,O,H` are the pre-, post-, and optional inhibitor edges. In an `Ematrix` term (`Tmatrix < Ematrix`) of the form `[_,_,_] f: F`, the adjacency lists are followed by flush arcs (subterm `F`; see the example below).

PT nets are defined modularly using AC juxtaposition `_;_ : Net Net -> Net` (identity `emptyNet`) and the subsort relation `Tran < Net`, both inherited (after renaming) from the `MAP` module.[1] The module `PT-SYS` extends the PT signature with a marking: a term `System` is the juxtaposition of a `Net` and a `Pbag`. Separating PT nets from PT systems improves reuse. The operators for transition enabling and firing redirect to the homonymous ones in `EMATRIX{PL :: TRIV}` (excerpt below). The enabling operator is partial: it applies only to well-formed terms where a place cannot be both ordinary and flush input (or output), and every flush output place has a corresponding input.

```
var E : Ematrix . vars Q Q' : Tmatrix . var F : FlushArcs . var M : Pbag .
op enabled : Ematrix Pbag -> [Bool] . *** partial function
ceq enabled(Q f: F, M) = I(Q) <= M and-then H(Q) > M and-then enabled(F, M)
    if well-def(Q f: F) .
op firing : Ematrix Pbag -> Pbag .
ceq firing(Q f: F, M) = M + O(Q) + O(Q') - I(Q) - I(Q') if Q' := flushInOut(F, M) .
```

[1] Importing a module in a *generated-by* mode means that no additional data values are created.

The SPN signature (SPN-SIG) integrates stochastic parameters into PT-NET: Transition labels include a String (tag), a Float (rate parameter), and a Nat (firing policy).

```
fmod SPN-SIG{PL :: TRIV} is
  protecting PT-NET{Tlab{String}, PL} . protecting CONVERSION .
  var M : Pbag . var T : Tran .
  op tag : Tran -> L$Elt . op rate : Tran -> Float . op pol : Tran -> Nat .
  eq tag(T) = tag(l(T)) .
  op firingRate : Tran Pbag -> [Float] .
  eq firingRate(T, M) = if pol(T) == 1 then rate(T) else rate(T) * float(if pol(T) == 0
       then enabDegree(I(T), M) else min(pol(T), enabDegree(I(T), M)) fi) fi .
  ...
endfm

fmod SPN-SYS{PL :: TRIV} is *** marked SPN
  protecting SPN-SIG{PL} . protecting PT-SYS{Tlab{String}, PL} .
endfm
```

For example, this term from SPN-SIG{Nat}

```
t("a",1.5,0) |-> [2 . p(2), 1 . p(2), 2 . p(1)] f: p(3) >> p(4,7)
```

describes a transition with tag "a", rate parameter $\mu = 1.5$, and firing policy 0. This transition requires at least two tokens in p_2, at least one in p_3, and at most one in p_1. In addition, it cannot exceed the capacity of $p_{(4,7)}$ (7). When it fires, it removes one token from p_2 and all tokens from p_3, moving the latter to $p_{(4,7)}$. The operator firingRate defines state-dependent rates based on the *enabling degree* (*ed*), i.e., the number of instances that can fire simultaneously. Under the infinite-server policy (0), the firing rate is $ed \cdot \mu$; under the k-server policy, with $k > 0$, it is $\min(ed, k) \cdot \mu$.

The system module SPN-EMU extends the stateful PT signature with the rule firing, which defines the marking change and the transition rate while considering place capacities. The seemingly redundant free variable R (bound to firing(T, M) via a matching equation) is used in [10] to identify all significant rule's matches.

```
mod SPN-EMU{PL :: TRIV} is
 including SPN-SYS{PL} .
 vars M M' : Pbag . var T : Tran . vars N N' : Net .
 crl [firing] : N M => N M' if T ; N' := N /\ enabled(T, M) /\
  M' := firing(T, M) /\ not(overCap(M')) /\ R:Float := firingRate(T, M) .
endm
```

An RwSPN is characterized by a module M comprising SPN-EMU and two constants, net0 and m0, for the underlying PT net and its initial marking; M may also specify other rules that modify the PT net structure.

2.1 Exploiting symmetries in RwSPN

The methodology in [13] extends rewritable PT nets with process-algebra-like compositional operators, simplifying modeling and enabling symmetry-based reduction for large nested models. A compact *quotient* transition system (TS) is then built through structured node labeling; it is strongly bisimilar to the standard TS and yields *exact lumpability* of the associated CTMC [7]. We summarize these ideas here.

1. A compact set of *net operators* supports modular construction of any `System` (`Net`) component. These operators implicitly encode symmetries in nested components through incrementally built place labels—lists of pairs `< String ; Nat >` defined in the `SLAB` module. The SPN signature is parameterized using this labeling scheme via the standard instantiation `SPN-SIG{Slab}`.
2. Symmetry reduces to a bipartite graph *morphism* ($\cong$) that preserves transition labels and markings. A morphism from a net to itself is an *automorphism*. The place labels encode the hierarchy of replica components. By convention, the hierarchy root is the label suffix: in a *well-labeled* `Net`, any two distinct maximal sets of places whose labels share the same (possibly empty) suffix, preceded by identical string pairs, are automorphic. For example:
   `{p(..< "S1" ; 1 >)}`$\cong$`{p(..< "S1" ; 2 >)}` (i)
   `{p(..< "S2" ; 1 > < "S1" ; 1 >)}`$\cong$`{p(..< "S2" ; 3 > < "S1" ; 1 >)}` (ii)
   In case (i), the two automorphic sets of places are associated with distinct instances of the top-level subsystem `"S1"`; in case (ii), they are associated with analogous components within a single `"S1"` subsystem.
3. Well-labeled `System` terms can be efficiently normalized by minimizing the `Pbag` subterm via index permutation on place labels (without backtracking), combined with name (index) abstraction, which makes label indices consecutive integers from zero at each nesting level. This normal form is the *least* with respect to a total order.
4. PT systems built via compositional operators are well-labeled by construction. Rewriting rules must preserve this property; hence, they are given in a "parametric" style and rely on suitable net-manipulation operators provided by the library [13].

Technically, we must only encapsulate the right-hand side of each rule (assumed to be of `System` type) in a suitable *normalization* operator. The strong bisimulation of the TS quotient generated by a normalized term (`net0 m0`) coincides with "exact lumpability"[2] in the associated CTMC [11], ensuring that states within an aggregate have equal probabilities. Correctly computing cumulative transition rates in the lumped CTMC requires handling the fact that a normalized term $\hat{s}$ can transition to multiple states $s', s'', \ldots$ with the same normal form $\hat{s}'$, which the standard TS generated by Maude cannot detect (notably when a rule has several matches). [10] accurately computes these rates through automated translation of the rules into nondeterministic operators.

We evaluated our methodology in a gracefully degrading production system [13] with N parallel replicas of a production line (PL), each with K mutually interchangeable components subject to controlled degradation. Upon faults, each PL autonomously reconfigures to remain partially operational (see [13,11]). The initial configuration is built by applying the core operator `repl&share`, which creates replicas that may share some places, nested. (https://github.com/lgcapra/rewpt/blob/main/modSPT/FTPL.maude)

In the following excerpt, `cycle` and `fault` denote the subnets that represent the basic production cycle and the fault process, respectively.

```
eq PL(K) = repl&share(cycle ; fault, K, "L", p (< "o" ; 0 >) U p(< "s" ; 0 >)) .
eq NPL(N, K) = repl&share(PL(K), N, "PL", p(< "s" ; 0 >)) .
```

[2] An equivalence relation in this CTMC is exactly lumpable [7] if, for each pair of equivalence classes, the sums of the transition rates from any state in one class to each state in the other (or the same) class are equal.

N	# states (quotient)	time (sec)	# states (conventional)	time (sec)	$R(10.000)$
2	451	0	1915	0	0.68
4	2123	3	124,148	21	0.79
6	4855	10	4,587,426	1530	0.86
8	8643	29	-	†	0.89
10	13487	66	-	†	0.90

Table 1: Conventional vs quotient TS – $R(t) := P(\text{TimeToAbsorption}) > t$ – † time out

Table 1 reports the results for this benchmark as the number of PL replicas grows, the system reliability at a fixed time instant is included (the CTMC has absorbing states). Without symmetry exploitation, the analytical solution is only possible for small setups.

3 Efficient multiformalism modeling

Modeling realistic systems with a single formalism is often inconvenient or infeasible. Multiformalism modeling [18,30,5,14,31,21] addresses this by using the most suitable formalism for each component while preserving coherence and exploiting modularity and formalism-specific features (e.g., *multisolution*). OsMoSys [17] and SIMTHESys [20,4] exemplify this approach: OsMoSys orchestrates workflows [25] that invoke external tools for different formalisms, while SIMTHESys defines custom formalisms via formalism-specific behavioral elements that generate dedicated evaluation tools.

Current multiformalism techniques offer limited support for dynamic modification of system components. The use of rewriting systems (especially Maude) in this context was first explored in [3,12]. We integrate the Maude-based multiformalism framework [3,12] by extending the compositional approach for rewritable SPNs [11], summarized in the previous section, to handle the analytical complexity of reconfigurable multiformalism models. In Section 4, we demonstrate the method in a case study.

3.1 A Maude framework for reconfigurable multiformalism

The multiformalism encoding in `Maude` relies on a compact, extensible module hierarchy, using advanced module operations to maximize the reuse of existing formalism encodings. We first present the generic core of the framework (later extended to support the compositional methodology based on structured labeling), then we present the components of the case study, one of which in detail. A complete account is available in the online repository at https://github.com/lgcapra/rewpt/tree/main/multiformalism.

Multiformalism framework The framework improves [3] by better describing the dynamics of nodes in heterogeneous networks, easing modeling, and allowing the focus to be on structural adaptation. A multiformalism model integrates various components (e.g., Petri nets, queueing networks, fault trees) that share a distributed state. This state is formalized by the theory of commutative monoids (`C-MONOID`), which defines a sort `Elt` and an AC juxtaposition operation, and serves as the parameter of the functional module `NETWORK{S::C-MONOID}` that provides the ADT of a multiformalism model.

The abstract syntax uses sort `Network`, its subsort `Node`, and an AC juxtaposition operator on `Networks`, written `_,_`, with identity `emptyNetW`. Thus, a multiformalism

model is a multiset (commutative monoid) of possibly heterogeneous `Network` components (nodes). Declaring suitable subsorts for node types yields a model of distinct components interacting through shared state. A term `NetSys` consists of a `Network` and a subterm of formal sort `S$Elt`, representing a *network* of interconnected nodes with a distributed *state*. Intuitively, a network of nodes n_i with shared state S is

```
n1, n2,..., nk : S
```

In our example, network nodes use a state representation analogous to the *marking* in PT nets, i.e., a multiset of places compactly encoded using the `PBAG{PL :: TRIV}` module. The AC sum `_+_` matches the expected syntax of `C-MONOID`. The module's parameter denotes the place label; this flexibility in choosing place labels is essential.

Using the parameterized view `S-Pag{PL::TRIV}`, we partially instantiate the parameter of `NETWORK{S::C-MONOID}` as a multiset of places (integer state variables), yielding `NETWORK-M{PL::TRIV}`. Varying the view gives alternative state notions, and its parameter fixes the label types later attached to places.

```
fth C-MONOID is
   sort Elt .
   op 0 : -> Elt .
   op _+_ : Elt Elt -> Elt [assoc comm id: 0] .
endfth

fmod NETWORK{S :: C-MONOID} is
 protecting EXT-BOOL .
 sorts Node Network NetSys .
 subsort Node < Network .
 op emptyNetW : -> Network [ctor] .
 op _,_ : Network Network -> Network [ctor assoc comm prec 123 id: emptyNetW] .
 op _:_ : Network S$Elt -> NetSys [ctor prec 125] .
 op netw : NetSys -> Network .
 op state : NetSys -> S$Elt .
 vars N N' : Network . var M : S$Elt .
 eq netw(N : M) = N .
 eq state(N : M) = M .
 op in : Network Network -> Bool .
 eq in((N, N'), N) = true .
 eq in(N, N') = false [owise] .
endfm

view S-Pbag{PL :: TRIV} from C-MONOID to PBAG{PL} is
 sort Elt to Pbag .
 op 0 to nilP .
endv

fmod NETWORK-M{PL :: TRIV} is
   protecting NETWORK{S-Pbag{PL}} .
endfm
```

Components used in the case-study Three node classes are considered: SPN extended with bounded places and flush I/O edges; (multi-class) Queueing Networks (MQNs),

used as reservoirs of elementary queues; and Join-the-Shortest-Queue (JSQ) nodes. The Markov-modulating process is modeled as a special SPN.

E-SPN We use the signature introduced in Section 2. The Buffer (Fig. 3a) and the modulating CTMC (Fig. 2) are modeled as SPNs, the latter with transition adjacency matrices `[1 . P, 1 . P' + 1 . s-t, 1 . s-t]`. Here, `s-t` is a special place shared by all SPN transitions, which is both an output and an inhibitor: when `s-t` is not marked, the transition from state `P` to `P'` pauses the CTMC evolution until the (quasi) instantaneous reconfiguration is complete.

Queue Networks We reuse the multi-class queue networks (MQNs) signature (not reported here). MQNs are sequences of `Server` elements (module `SERVER{PL::TRIV}`), written `P @ F` for a `Place` and exponential service rate `Float`. The MQN signature (`QUEUE{PL::TRIV}`) is based on `LIST{X::TRIV}`. A simple MQN (sort `SimpleQ`) is a non-empty list of servers (`NeList{Server}`) ending in a `Place`, written `NeL > P`. Elementary queues (sort `ElQueue`) contain a single server, and in general an MQN (sort `Queue`) is a `SimpleQ` with two `Pbag` terms (in `[_,_]`) for the input and inhibitor enabling conditions, i.e. `ElQueue < SimpleQ < Queue`. The operator `queueRate` returns the service rate at each MQN stage as a function of the global population. Within a `Network`, MQNs are composed by juxtaposition (`_,_`) and shared places. In the edge-server example, we build pools of homogeneous elementary queues for the CPU (Fig. 3b)) and GPU (Fig. 3d)) under different configurations. The Network component (Fig. 3c)) has two queues, and the Storage (Fig. 3e)) has one.

JSQ (Join the Shortest Queue) This component plays a pivotal role in the allocation of jobs to a group of queues according to the eponymous algorithm. The JSQ signature is provided below. A term of type `Jsq` is specified by an input place and a (possibly empty) set of output places that are entry points for its queues, written `P |> S`, where `S` is a set of places. The `jsqRate` operator defines state-dependent scheduling times.

```
fmod JSQ{PL :: TRIV} is
  protecting PSET{PL} . protecting FLOAT .
  var P : Bplace . var S : Pset . var L : PL$Elt . var I : Nat .
  sort Jsq .
  op _|>_ : Bplace Pset -> [Jsq] [ctor] . *** partial function ...
 cmb p(L, 1) |> S : Jsq if p(L, 1) in S = false . *** JSQs input places must have capacity 1
  op In : Jsq -> Bplace .
  eq In(P |> S) = P .
  op Out : Jsq -> Pset .
  eq Out(P |> S) = S .
  op newJsq : Bplace -> [Jsq] .
  eq newJsq(P) = P |> emptyPset .
  op newJsq : PL$Elt -> Jsq .
  eq newJsq(L) = p(L, 1) |> emptyPset .
  op jsqRate : Jsq Pbag -> Float .
  ...
endfm
```

3.2 Efficient analysis of multiformalism models

Multiformalism models of realistic scenarios often become too complex for analytical solutions. We address this intractability by extending the symmetry-based methodology for RwPNs [13] to the just introduced multiformalism framework. This method should be combined with other techniques—such as abstractions, stochastic approximations (e.g., PN transition aggregations), and hierarchical multi-level analysis—but it remains a necessary step for analytical or simulation-based approaches.

As before, we extend existing hierarchies, prioritizing reuse. We present a modular architecture parallel to Section 3.1 that incrementally enriches network-node labels through compositional operators. We first outline the generic architecture and then instantiate it for JSQ; the same pattern applies to each class of interconnected nodes.

The approach is based on the flexible labeling scheme of Section 3.1. The module `NETWORK{S :: C-MONOID}` is first partially instantiated in `NETWORK-M{PL :: TRIV}` via the view `S-Pbag`. A further instantiation with `Slab`, mapping `TRIV` to `SLAB`, yields structured labels such as pairs lists `< String ; Nat >`. The resulting module expression `NETWORK-M{Slab}` defines a network of components with state distributed over places, where the places are annotated with these structured labels.

Compositional multiformalism models: core generic part This part contains parameterized modules and views. Models a generic heterogeneous network as an *extended* monoid whose elements (nodes) support a small set of primitive operators to efficiently manipulate structured labels. This is captured by the theory `MO-NET`, based on `MONOID+`, a monoid with a subsort `El < Elt`, where `El` denotes base elements. The module `MO-NET-OP`, parameterized by `MO-NET`, defines a concise set of generic operations for manipulating and normalizing any monoid-based model with node-structured labels, systematically built on the node-level operators of `MO-NET`.

Some operations (e.g. `replaceWith`, `places`) support the modular normalization algorithm for network terms, while others (e.g. `addLab`) are used by the core compositional operator `repl&share`. For example, the excerpt from `MO-NET-OP` shows how `replaceWith`—which performs index substitution throughout the entire monoid-like network—is defined in terms of `replaceWithEl`, which is formally specified in `MO-NET` and operates at the level of individual nodes. Network elements are essentially collections of places, and a place-based notion of state is used, but the method easily generalizes to arbitrary structures and state notions.

The theory `MO-NET-SYS` extends `MO-NET` to describe heterogeneous networks of nodes sharing a distributed state (a multiset of places) via the juxtaposition of subterms, forming the sort `MoNetSys`. It serves as the formal parameter of `MO-NET-SYS-OP`, which extends `MO-NET-OP` with primitive operations to efficiently manipulate such networks. Normalization and name-abstraction algorithms lift directly, and additional mechanisms are provided, such as purging a state (removing non-encoded places) and associating a state with a network subterm. Advanced module operations include the view homonym of `MO-NET-SYS`, embedding `MO-NET` and thereby enabling `MO-NET-OP` (which requires a `MO-NET` parameter) to be imported into `MO-NET-SYS-OP`. We summarize the main features of this methodology. Some code excerpts are then reported to illustrate them.

1. Any subsystem that admits a structural characterization as a monoid can be specified systematically by encapsulating the `MO-NET-OP` module—or, when a state is present, the `MO-NET-SYS-OP` module—according to a well-established and simple schema.
2. This property holds uniformly at all abstraction levels: globally (network-wide), for individual components (e.g., a Stochastic Petri Net can be formalized via this schema) and within components, whenever suitable monoid substructures exist. This enables a simple, modular definition of hierarchies of cooperating components that undergo uniform, consistent normalization.
3. Foundational data structures such as sets, multisets, and lists—common in modeling—can be treated as monoids and represented in this schema, allowing homomorphic operations on the structured labels of network node places. For example, module `PBAG-MOD` (not reported here), which defines a multiset of places with structured labels, encapsulates `MO-NET-OP` and is included in theory `MO-NET-SYS` (and thus in `MO-NET-SYS-OP`) and in the SPN specification `SPN-SYS-MOD`. Similarly, module `PSET-MOD` defines sets of places, is based on `MO-NET-OP`, and is reused throughout the framework.
4. As noted previously, we use the same essentially normalization strategy as in [13,11] (Section 2.1). This converts a `N$MoNetSys` term into a minimal form determined by a total lexicographic order. It combines name (index) abstraction in basic network components (`abstract` in `MO-NET-OP` and `MO-NET-SYS-OP`) with a state-minimization algorithm based on simple index permutation without backtracking (operator `minimize` in `PBAG-MOD`). Its efficiency relies on well-structured labels generated by compositional operators (`repl&share` in `MO-NET-OP`), a property that must be preserved by all rewrite rules. Unlike [13], this method is fully general: it applies recursively to any monoid-like structure, encapsulating `MO-NET-SYS-OP` or `MO-NET-OP` depending on whether a component has a distributed or local state.
5. The methodology in [13] (Section 2.1) uses a hierarchical symmetry notion that, for PT nets, coincides with bipartite (sub)graph morphisms. In a multiformalism setting, this must be refined for heterogeneous node structures: in `JSQ` nodes, symmetry is given by morphisms on (sub)sets, while in `QUEUE` nodes it uses morphisms on (sub)lists. Intuitively, we say

   `n1, n2,..., nk : S` $\cong$ `n1', n2',..., nk' : S'`

   iff there exists a family of morphisms $\phi = \{\phi_i \mid i{:}\ldots k\}$ such that $\phi_i(n_i) = n'_i$ and $\phi(S) = S'$ (the ordering of the nodes n_i is inconsequential). We call ϕ an automorphism if each ϕ_i is an automorphism.
6. Labels on structural elements forming the nodes (places here, though other types are possible) explicitly capture the model hierarchy and implicitly encode its symmetries. In a *well-labeled* `Network`, any two distinct maximal sets of elements whose labels share the same suffix (possibly empty) and are preceded by the same sequence of label pairs are related by an automorphism. We assume compositional operators enforce well-labeling, and rewriting rules preserve it. Thus, the TS quotient induced by a stateful network term `n1, n2, ..., nk : S` is exactly lumpable.

Presented below are several excerpts from the general framework that offers a monoid-style characterization of heterogeneous network components (as well as data structures).

```
fth MONOID+ is
 sorts El Elt . subsort El < Elt .
 op nil : -> Elt .
 op _;_ : Elt Elt -> Elt [assoc id: nil] .
endfth

fth MO-NET is
  including MONOID+ . including PSET-LAB .
  op replaceWithEl : El NeLab Nat -> El .
  op addLabEl : El Lab Pset -> El .
  op placesEl : El List{String} -> Pset .
  op placesEl : El Lab -> Pset .
endfth

fmod MO-NET-OP{N :: MO-NET} is
 var J : Nat . vars K K' : NzNat . vars N N' : N$Elt . var E : N$El .
 vars L L' : Lab . var NeL : NeLab . vars W W' : String . var P : Place .
 vars S S' : Pset . var WL : List{String} . var NeWL : NeList{String} .
 op replaceWith : N$Elt NeLab Nat -> N$Elt .
 eq replaceWith(N, NeL, J) = $replaceWith(N, NeL, J, nil) .
 op $replaceWith : N$Elt NeLab Nat N$Elt -> N$El .
 eq $replaceWith(nil, NeL, J, N) = N .
 eq $replaceWith(E ; N,NeL,J,N') = $replaceWith(N,NeL,J, N' ; replaceWithEl(E,NeL,J)) .
 op addLab : N$Elt Lab Pset -> N$Elt .
 op places : N$Elt List{String} -> Pset .
 op places : N$Elt NeLab -> Pset .
 ...
 op in : N$Elt NeLab -> Bool . *** looks for a place with a label prefix
 ceq in(E ; N , NeL) = true if placesEl(E, NeL) =/= emptyPset .
 eq in(N, NeL) = false [owise] .
 op abstract : N$Elt -> N$Elt . **** "name" (index) abstraction
 ceq abstract(E ; N) = abstract(replaceWithEl(E, < W ; K > L, J) ;
  replaceWith(N , < W ; K > L, J)) if P U S := placesEl(E) /\
  L' < W ; K > L := lab(P) /\ J := sd(K,1) /\ not(in(N,< W ; J > L)) .
 eq abstract(N) = N [owise] .
 op repl&share : N$Elt NzNat String Pset -> N$Elt . *** replica of a node
 eq repl&share(N, K, W, S) = $repl&share(N, K, W, nil, S) .
 op $repl&share : N$Elt Nat String N$Elt Pset -> N$Elt .
 eq $repl&share(N, 0, W, N', S) = N' .
 ceq $repl&share(N,K,W,N',S) = $repl&share(N,J,W,(N' ; addLab(N,< W ; J >,
     S)), S) if J := sd(K,1) .
 op replica : N$Elt NzNat String -> N$Elt . *** default
 eq replica(N, K, W) = repl&share(N, K, W, emptyPset) .
endfm

fth MO-NET-SYS is
 including MO-NET . including PBAG-MOD .
 sort MoNetSys . *** stateful network
 op _:_ : Elt Pbag -> MoNetSys .
endfth
```

```
view MO-NET-SYS from MO-NET to MO-NET-SYS is endv *** target is a theory

fmod MO-NET-SYS-OP{N :: MO-NET-SYS} is
 protecting MO-NET-OP{MO-NET-SYS}{N} . protecting PSET-MOD .
 var J : Nat . var K : NzNat . var M : Pbag . var P : Place .
 var W : String . var L : Lab . var NeL : NeLab . var S : Pset .
 vars Sys Sys' : N$MoNetSys . var N : N$Elt .
 op nd : N$MoNetSys -> N$Elt . *** node structure
 op st : N$MoNetSys -> Pbag . *** node shared state
 eq nd(N : M) = N .
 eq st(N : M) = M .
 op replaceWith : N$MoNetSys NeLab Nat -> N$MoNetSys .
 eq replaceWith(N : M, NeL, J) = replaceWith(N, NeL, J) : replaceWith(M,
     NeL, J) .
 op abstract : N$MoNetSys -> N$MoNetSys .
 ceq abstract(N : M) = abstract(replaceWith(N : M, < W ; K > L, J))
    if P U S := places(N) /\ L' < W ; K > L := lab(P) /\ J := sd(K,1) /\
     not(in(S, < W ; J > L)) .
 eq abstract(Sys) = Sys [owise] .
 op normalize : N$MoNetSys -> N$MoNetSys .
 ceq normalize(Sys) = nd(Sys') : minimize(st(Sys'))
    if Sys' := abstract(Sys) .
 ...
endfm
```

3.3 Using the framework to exploit the symmetries of multiformalism models

We extend the multiformalism framework of Section 3.1 to exploit symmetries in monoid-like structures. We first consider the network level and then the component level.

The module NETWORK-LAB instantiates NETWORK-M through view Slab, giving structured labels to basic components, and imports PSET-MOD and PBAG-MOD, which define monoidal (multi)set structures of places with structured labels. It then provides an abstract specification of the operations required by theory MO-NET-SYS, reusing its operation names; these are later instantiated when concrete components are linked to the network. The view NETWORK-LAB maps MO-NET-SYS to the target module with default operator mappings, assigning (sub)sort El, the basic monoid elements, to Node, representing the network nodes. Finally, NETWORK-MOD encapsulates NETWORK-LAB within MO-NET-SYS-OP via the homonymous view, thus treating NETWORK-LAB as a monoid equipped with the operations needed for compositional modeling. Modules PBAG-MOD and PSET-MOD are instantiated from parameterized modules PBAG{PL::TRIV} and PSET{PL::TRIV}, respectively, and follow the same schema while wrapping the stateless monoid structure MO-NET-OP via views from theory MO-NET.

```
fmod NETWORK-LAB is
 protecting PBAG-MOD . protecting PSET-MOD . protecting NETWORK-M{Slab} .
 op replaceWithEl : Node NeLab Nat -> Node . *** formal operations
 op addLabEl : Node Lab Pset -> Node .
 op placesEl : Node List{String} -> Pset .
```

```
 op placesEl : Node Lab -> Pset .
endfm

view NETWORK-LAB from MO-NET-SYS to NETWORK-LAB is
 sort Elt to Network . sort El to Node . sort MoNetSys to NetSys .
 op nil to emptyNetW .
 op _;_ to _,_ .
endv

fmod NETWORK-MOD is
 protecting MO-NET-SYS-OP{NETWORK-LAB} .
endfm
```

Linking nodes to the `Network` A simple pattern connects any node type; we illustrate it with JSQ nodes. We define a functional module (`JSQ-NODE`) that bundles the node signature (`JSQ`) and the network ADT (`NETWORK-M`), instantiated by declaring the concrete node sort (`Jsq`) as a subsort of `Node`. For convenience, this module is wrapped in a system module (`JSQ-NODE-SYS`), which specifies the node's network behavior via a rewrite rule. To integrate heterogeneous components into an interacting network with shared distributed state and possible behavioral symmetries, we apply to each component a modeling pattern analogous to that used at the overall network level.

The next code excerpt applies this procedure to the JSQ component. From the parameterized module `JSQ`, we build a wrapper module `JSQ-MOD` that instantiates `JSQ` with structured place labels and imports `PSET-MOD`, since a JSQ is essentially a set of places. Another module then combines `JSQ-MOD` with `NETWORK-MOD`: the abstract operations of `NETWORK-MOD` (from theory `MO-NET`) receive concrete definitions for `Jsq` via subsort overloading from `Jsq < Node`, mapped directly to the corresponding operations in `PSET-MOD`. We apply the same pattern to Queue nodes, starting from `QUEUE`: a `Queue` is represented as a list of servers embedded in a monoid structure, analogous to the treatment of sets and multisets of places.

The SPN component uses a slightly modified version of `SPN-SIG`. An SPN is treated as a monoid (with `Tran` terms as elements and `_;_` as the associative operation) and embedded in `MO-NET-OP` via a suitable view, yielding a recursive structure where the heterogeneous network and some nodes are monoids. When SPN nodes must interconnect while keeping local state, we use the stateful monoid representation in `MO-NET-SYS-OP`.

```
fmod JSQ-MOD is
 protecting JSQ{Slab} . protecting PSET-MOD . *** set defined as a monoid
endfm

fmod JSQ-NODE-MOD is
  protecting JSQ-MOD . extending NETWORK-MOD .
  subsort Jsq < Node .
  var L : Lab . var NeL : NeLab . var P : Place . var S : Pset . var I : Nat .
  var Ls : List{String} .
  eq replaceWithEl(P |> S,NeL,I) = replaceWith(P,NeL,I) |> replaceWith(S,NeL, I) .
  eq addLabEl(P |> S, L, S) = addLab(P, L, S) |> addLab(S, L, S) .
  eq placesEl(P |> S, Ls) = places(S U P, Ls) .
```

```
  eq placesEl(P |> S, NeL) = places(S U P, NeL) .
  eq placesEl(P |> S) = S U P .
endfm

mod JSQ-NODE-SYS-MOD is
 including JSQ-NODE-MOD .
 var N : Network . var B : Pbag . var P : Bplace . var P' : Place .
 var NeS : NePset . var S : Pset .
 crl (N , P |> NeS) : 1 . P + B => (N , P |> NeS) : normalize(B + 1 . P')
    if P' U S := emin(B, NeS) /\ rate:Float := jsqRate(P |> NeS, B) .
endm
```

4 Case study: an edge server in a smart city

We consider a smart city case study, detailed and analyzed in [23]. Our goal is to assess the effectiveness of our symmetry-based approach. We present a small selection of quantitative results and refer to [23] for a more detailed analysis.

In smart cities, sensor data are continuously collected using ICT such as IoT and edge computing to monitor conditions and enable adaptive solutions for future traffic, behavior, events, transportation demand, and needs [2,19,16]. We consider an edge server that gathers sensor data and stores them on a blockchain [8], facing a variable workload as roadside sensors scale with vehicle density and cars move in and out of coverage. The server includes data acquisition, computing, networking, and temporary storage. A multi-core processor aggregates data into packets and writes them to the blockchain, pausing when the network is unavailable or power is low. If a watchdog is activated or storage exceeds a threshold, a backup radio is enabled to process and store buffered data. The edge server usually runs in energy-saving mode, with backup radio, storage, and extra CPU capacity off, scaling CPU utilization between 50% and 80%.

The nominal structure of the edge server is shown in Fig. 1, and the overall adaptive behavior of the system follows the state diagram in Fig. 2. The states L_0–L_3 represent increasing load levels, with transitions from L_i to L_j at rate q_{ij}, forming a canonical Markov-modulated process.

- L0: Buffer active (only here), 1 CPU core for computation+encryption, slow network (A) only. Buffer supports blockchain, which is more efficient on large datasets.
- L1: 3 CPU cores for computation+encryption, fast network (B) only.
- L2: 6 CPU cores, both networks (A+B), CPU only for computation, 1 GPU board for encryption.
- L3: all 8 CPU cores, both GPUs for encryption, both networks (A+B), and Storage.

Fig. 1 shows the functional blocks; Fig. 2 shows the allowed adaptations and active hardware. Fig. 3a) alternates two submodels: when the buffer is inactive, the lower submodel is omitted; otherwise, the upper PT models a buffer where packets in place n wait until packets accumulate K (enabling $t_{collect}$) or the timeout $T_{timeout}$ fires. The packets then move to the place m and to the CPU via $t_{continue}$; the dashed "flush" edges encode this. Fig. 3b) shows two related CPU submodels, the second parametric: the upper has a single queue CPU_1; the lower, parameterized by N, models $N > 1$ active cores with queues CPU_i ($1 \leq i \leq N$), with arrivals routed by JSQ. Fig. 3c) alternates

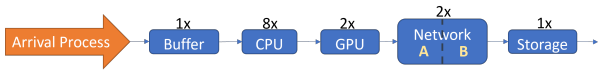
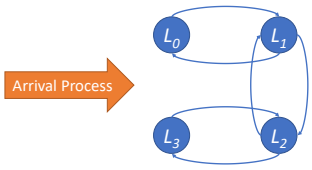


Fig. 1: The nominal model of the scenario considered.



	Buf	CPU	GPU	Net	Stor
L_0	1	1	0	A	0
L_1	0	3	0	B	0
L_2	0	6	1	A+B	0
L_3	0	8	2	A+B	1

Fig. 2: The MC-modulated arrival process and the corresponding configurations.

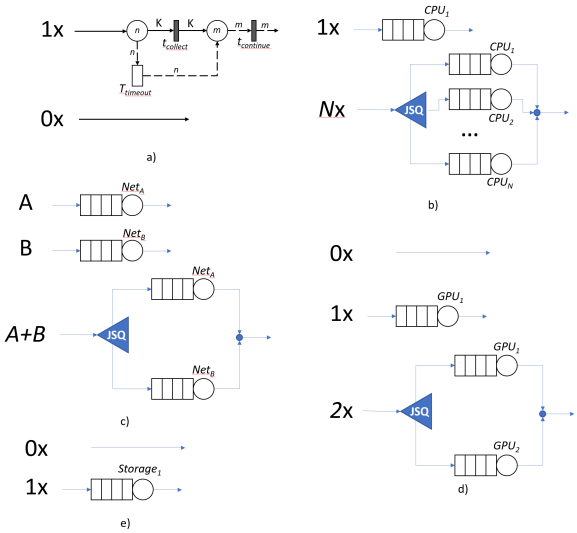


Fig. 3: a) the *Buffer*; b) the *CPU*; c) the *GPU*; d) the *Network*; e) the *Storage*.

two GPU submodels, mirroring the CPU case; when the GPU is used, the CPU demand decreases. Fig. 3d) has three operational submodels: only module A (queue Net_A), only B (queue Net_B) and both A and B (analogous to the 2-GPU case). Fig. 3e) offers two alternatives, with or without active storage: in the first, storage is absent; in the second, it is modeled by queue $Storage_1$.

4.1 Formalization and analysis of the case study

The case study is divided into several system modules. The core (listed below) includes three modules `<NTYPE>-NODE-SYS-MOD` that characterize the dynamics of each type of node present in the network, as explained. It also includes a fourth system module, `MC-EMU`, which formalizes the Markov modulating process as a specific stochastic Petri net (SPN). This module encapsulates `SPN-EMU` by redefining selected sorts and operators to avoid naming conflicts. Transitions are directly annotated with their rate parameters.

The CTMC component is intentionally left unconnected to `Network` because it acts as a meta-controller of the evolution of the network. (Maude provides meta-level modules, but we found it more convenient to represent this at the object level.)

```
mod MM-BLOCKCHAIN is
 including QUEUE-NODE-SYS-MOD . including SPN-NODE-SYS-MOD{String} .
 including JSQ-NODE-SYS-MOD . including MC-EMU .
 sort MMnetwork . *** markov modulated network
 op MM:_N:_ : System NetSys -> MMnetwork [ctor] .
 op mc : MMnetwork -> System .
 op sys : MMnetwork -> NetSys .
 var MC : System . var Sys : NetSys .
 eq mc (MM: MC N: Sys) = MC .
 eq sys(MM: MC N: Sys) = Sys .
endm
```

Through the constructor `op MM:_N:_ : System NetSys -> MMnetwork` we represent CTMC and network as distinct components, with a term `MMnetwork` denoting their amalgamation. The `System` subterm denotes a CTMC along with its initial state (given as a singleton multiset of places). The system module `MC-EMU` contains the SPN firing rule, which triggers the local rewrite encoding a state transition of the CTMC. Two additional modules complete the formalization of the case-study by defining compositional operators and rules for Markov-chain–driven network reconfiguration (details in https://github.com/lgcapra/rewpt/tree/main/multiformalism/MODmulti/BLOCKCHAIN-MOD.maude). In particular, the operators:

```
op pool : ElQueue NzNat -> Network
op connect&link : Jsq Network -> Network
```

build a pool of similar elementary queues with a single exit point and connect them to a JSQ. The first relies on the *symmetric* composition operator `repl&share`; the second is derived from the first. Such operators are crucial for scalability in both modeling and analysis. We conclude by listing a self-contained excerpt from the final system module (`BLOCKCHAIN-EXE`), showing the two rewriting rules that reconfigure the system between states L_0 and L_1 (Fig. 1). (Here, the transition rates are arbitrary; the precise stochastic parameters are in [23]).

```
op mmc : -> Q [memo] . *** modulating Markov chain
op L : Nat -> Place [memo] . *** states of the CTMC
eq L(i) = p(< "mc" ; i >) .
op s-t : -> Place [memo] .
eq s-t = p(< "s-t" ; 0 >) .
eq mmc = 0.01 |-> [1 . L(0); 1 . L(1) + 1 . s-t ; 1 . s-t] ;
0.02 |-> [1 . L(1); 1 . L(0) + 1 . s-t ; 1 . s-t] ;
0.02 |-> [1 . L(2); 1 . L(1) + 1 . s-t ; 1 . s-t] ; 0.01 |-> [1 . L(1); 1 . L(2) + 1 . s-t ; 1 . s-t] ;
0.02 |-> [1 . L(2); 1 . L(3) + 1 . s-t ; 1 . s-t] ; 0.01 |-> [1 . L(3); 1 . L(2) + 1 . s-t ; 1 . s-t] .
op N : -> NzNat [memo] . eq N = 4 . *** model's parameter (place capacity)
ops emptyp pin cpuIn cpuOut gpuIn gpuOut netOut a b : NzNat -> Place [memo] .
eq emptyp(k) = p(emptyLab, k) . *** bounded place with empty lab
eq pin(k) = replica(emptyp(k), 1, "in") . *** bounded input place (symmetric style)
eq cpuIn(k) = replica(emptyp(k), 1, "cpu") .
```

```
eq cpuOut(k) = replica(emptyp(k), 1, "cpu-Out") .
...
var I : Nat . vars K K' K'' : NzNat . vars P P' : Place . var R : Float .
var MC : System . var Q : Q . vars S B : Pbag . var Nw : Network .
op buffer : -> Net [memo] .
eq buffer = newTin(pin(N),2.0) ; t("collect",1.0,1) |-> [N . pin(N), N . p(< "b" ; 0 >, N)] ;
      t("timeout", 0.2, 1) |-> [nilP, nilP] f: pin(N) >> p(< "b" ; 0 >, N) ;
      t("continue", 1.0, 1) |-> [nilP, nilP] f: p(< "b" ; 0 >, N) >> cpuIn(N) .
ops A B : Place NzNat -> ElQueue [memo] . *** parametric in the input and in the capacity
eq A(P, K) = newElQueue(P, netOut(K), 2.0) .
eq B(P, K) = newElQueue(P, netOut(K), 4.0) .
ops cpu gpu : NzNat NzNat -> Network [memo] . *** 1st par: replica, 2nd: capacity
eq cpu(K, K') = pool(newElQueue(emptyp(N), cpuOut(K'), 5.0), K, "cpu") .
op network : Nat -> [Network] [memo] .
eq network(0) = buffer, cpu(1, N), A(cpuOut(N), N), newTout(netOut(N), 1.0) .
eq network(1) = tin, connect&link(newJsq(pin(1)), cpu(3, N)), B(cpuOut(N), N),
  newTout(netOut(N), 1.0) .

crl [S0>S1] : MM: (Q S) N: (Nw : B) => MM: (Q 1 . L(1)) N: normalize((
    network(1) : B)) if S == 1 . L(1) + 1 . s-t /\ in(Nw, buffer) .
crl [S1>S0] : MM: (Q S) N: (Nw : B) => MM: (Q 1 . L(0)) N:
   normalize((network(0) : set(B, pin(cpu(1, N)), I)))
  if S == 1 . L(0) + 1 . s-t /\ in(Nw, cpu(3, N)) /\
   I := allClients(cpu(3, N), B) /\ not(I <= cap(pin(cpu(1, N)))) .
...
op MMsys0 : -> MMnetwork . *** initial setup
eq MMsys0 = MM: (mmc 1 . L(0)) N: (network(0) : nilP) .
```

We use aliasing to improve readability (e.g., operators `buffer`, `A`, `B`, `cpu`, `gpu` map to components in Fig. 3, and `mmc` denotes the modulating CTMC in Fig. 2). Specifically, `cpu(K, K')` denotes a CPU pool defined by compositional symmetric operators, and `network(I)` represents the system configuration in state L_I. We also use the symmetric operator `replica` to build correctly labeled places from an initially empty one since the places (module `PSET-MOD`) form a monoid. The rewriting mechanism is activated exactly when the CTMC place `s-t` holds one token (indicating a CTMC state transition) and is deactivated as soon as this token is removed from `s-t`. During the transition from L_1 to L_0, the number of CPUs decreases from three to one. In this phase, the policy may delay job migration based on the processing capacity of the remaining core, slightly violating the assumption of instantaneous reconfiguration. Two additional pairs of rules, defined analogously, formalize the transitions $L_1 \leftrightarrow L_2$ and $L_2 \leftrightarrow L_3$ (see Figure 2).

To construct the TS quotient, we wrap the `NetSys` subterm of `MMnetwork` on the right-hand side of the rules in the `normalize` operator. Table 2 reports the performance of the Maude command `search mcStateTran(MMsys0) =>! M:McStateTran`, which identifies final states in the fully reconfigurable edge-server model (`MMsys0` is the initial setup). A single-parameter model is used, with N as the capacity of most places; the searches show the system is deadlock-free. Without symmetries, analysis is feasible only for small setups ($N \leq 2$); normalization adds acceptable overhead, and for $N = 16$ the TS is estimated to contain tens of billions of states. The model still generates large state spaces as N grows, because only CPUs (and to a lesser extent GPUs) are

significantly symmetric, yielding only partial symmetry. Thus, additional techniques such as abstraction and structural reductions must complement symmetry-based reductions.

We use a parameter $0 < \alpha < 1$ for the modulation process. With base switching time $T_{base} = 6$ hours, we set $q_{i,i+1} = 1/(\alpha T_{base})$ and $q_{i,i-1} = 1/((1-\alpha)T_{base})$. The solution is obtained through discrete-event simulation using the R solver [32] on the lumped CTMC generator produced by preprocessing in [10]. Simulation runs are on average an order of magnitude faster than the corresponding build times in Table 2. Fig. 4 shows the response time and monthly cost (secondary axis) using the arrival rates of [23]. The former is acceptable unless the system is too long in L_0, where the buffer is used; the monthly cost increases by nearly an order of magnitude when the system is longer in L_3.

	TS		Quotient TS	
N	states(final)	time (sec)	states(final)	time (sec)
2	5,789,456(0)	2445	37,344(0)	40
4	-	†	303,534(0)	367
8	-	†	2,990,842(0)	4200
16	-	†	18,189,496(0)	27070

Table 2: Build time of conventional TS vs normalized TS as the capacity varies)

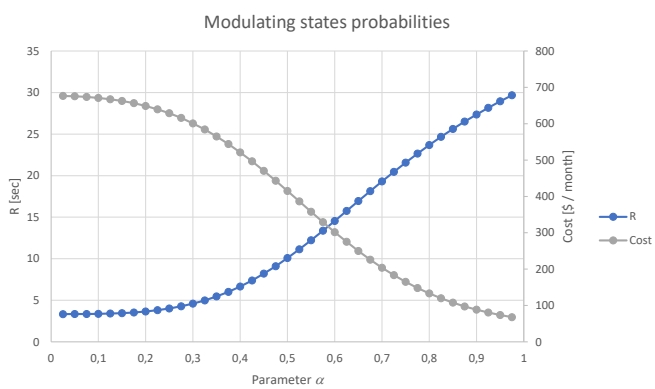


Fig. 4: Response times and costs for different values of the parameter α.

5 Conclusions

We present a unified technique for analyzing adaptive systems that combines two approaches. The first derives a lumped Markov process for RwSPNs using compositional operators that exploit symmetries via a structured node-labeling scheme reflecting model modularity. The second uses Maude as a multiformalism framework for models combining several formalisms. We extend compositional RwSPNs to this setting and integrate them into the preprocessing-based approach of [10] to derive a lumped CTMC, reporting promising initial results in an edge-computing case study. We plan to extend our methodology to encompass a larger variety of nodes—interpreted as fundamental building blocks of network components—as well as a broader spectrum of state concepts, including components with both private (hidden) and public (shared) state.

References

1. Agha, G., Meseguer, J., Sen, K.: Pmaude: Rewrite-based specification language for probabilistic object systems. Electronic Notes in Theoretical Computer Science **153**(2), 213–239 (2006). https://doi.org/https://doi.org/10.1016/j.entcs.2005.10.040, proceedings of the Third Workshop on Quantitative Aspects of Programming Languages (QAPL 2005)
2. Alshekhly, I.: Smart cities: Survey. J. Adv. Comput. Sci. Technol. Res **2**, 79–90 (2012)
3. Barbierato, E., Capra, L., Gribaudo, M., Iacono, M.: Enhancing multiformalism models with rewrite engines. Proceedings of EPEW 2025, June 26-27, 2025 Catania, Italy (2025)
4. Barbierato, E., Gribaudo, M., Iacono, M.: Modeling hybrid systems in SIMTHESys. Electronic Notes in Theoretical Computer Science **327**, 5 – 25 (2016). https://doi.org/10.1016/j.entcs.2016.09.021
5. Bause, F., Buchholz, P., Kemper, P.: A toolbox for functional and quantitative analysis of DEDS. In: Proc. of 10th Int. Conf. on Comp. Perf Ev.: Modelling Techniques and Tools. pp. 356–359. TOOLS '98, Springer-Verlag, UK (1998)
6. Bruni, R., Meseguer, J.: Generalized rewrite theories. In: Baeten, J.C.M., Lenstra, J.K., Parrow, J., Woeginger, G.J. (eds.) Automata, Languages and Programming. pp. 252–266. Springer-Verlag, Berlin, Heidelberg (2003). https://doi.org/10.1007/3-540-45061-0_22
7. Buchholz, P.: Exact and ordinary lumpability in finite markov chains. Journal of Applied Probability **31**(1), 59–75 (1994). https://doi.org/https://doi.org/10.2307/3215235
8. Campanile, L., Iacono, M., Mastroianni, M., Riccio, C.: Performance evaluation of an edge-blockchain architecture for smart city. vol. 2025-June, p. 620 – 627 (2025). https://doi.org/10.7148/2025-0620
9. Capra, L.: A Maude implementation of rewritable Petri nets: a feasible model for dynamically reconfigurable systems. In: Gleirscher, M., Pol, J.v.d., Woodcock, J. (eds.) Proceedings of the First Workshop on Applicable Formal Methods, 2021. Electronic Proceedings in Theoretical Computer Science, vol. 349, pp. 31–49. Open Publishing Association (2021). https://doi.org/10.4204/EPTCS.349.3
10. Capra, L.: Associating a Markov process with Maude executable modules. In: Proceedings of the 15th International Conference on Simulation and Modeling Methodologies, Technologies and Applications. pp. 106–116. SciTePress (2025)
11. Capra, L., Gribaudo, M.: A lumped CTMC for modular rewritable PN. In: Doncel, J., Remke, A., Pompeo, D.D. (eds.) Computer Performance Engineering - 20th European Workshop, EPEW 2024, Venice, Italy, June 14, 2024. Lecture Notes in Computer Science, vol. 15454, pp. 106–120. Springer (2024)
12. Capra, L., Gribaudo, M., Iacono, M., Köhler-Bußmeier, M.: Using rewriting systems for performance analysis. QualITA 2025: The Fourth Conference on System and Service Quality, June 25 and 27, 2025, Catania, Italy, Proceedings (2025)
13. Capra, L., Köhler-Bußmeier, M.: Modular rewritable Petri nets: An efficient model for dynamic distributed systems. Theoretical Computer Science **990**, 114397 (2024). https://doi.org/10.1016/j.tcs.2024.114397
14. Ciardo, G., Miner, A.S.: SMART: The stochastic model checking analyzer for reliability and timing. In: Quantitative Evaluation of Systems, Int. Conf. on. pp. 338–339. IEEE, Los Alamitos, CA, USA (2004)
15. Clavel, M., Durán, F., Eker, S., Lincoln, P., Oliet, N.M., Meseguer, J., Talcott, C.: All About Maude - A High-Performance Logical Framework: How to Specify, Program, and Verify Systems in Rewriting Logic. Lecture Notes in Computer Science, Springer (July 2007). https://doi.org/10.1007/978-3-540-71999-1
16. Far, A.Z., Far, M.Z., Gharibzadeh, S., Zangeneh, S., Amini, L., Rahimi, M.: Artificial intelligence for secured information systems in smart cities: Collaborative iot computing with deep reinforcement learning and blockchain. arXiv preprint arXiv:2409.16444 (2024)

17. Franceschinis, G., Gribaudo, M., Iacono, M., Mazzocca, N., Vittorini, V.: Drawnet++: Model objects to support performance analysis and simulation of systems. Lecture Notes in Computer Science **2324**, 233 – 238 (2002)
18. Gribaudo, M., Iacono, M.: An introduction to multiformalism modeling (2013). https://doi.org/10.4018/978-1-4666-4659-9.ch001
19. Guo, S., Qi, Y., Yu, P., Shao, S., Qiu, X.: Edge network resource synergy for mobile blockchain in smart city. In: 2020 International Wireless Communications and Mobile Computing (IWCMC). pp. 1272–1277. IEEE (2020)
20. Iacono, M., Gribaudo, M.: Element based semantics in multi formalism performance models. p. 413 – 416 (2010). https://doi.org/10.1109/MASCOTS.2010.54
21. de Lara, J., Vangheluwe, H.: Atom3: A tool for multi-formalism and meta-modelling. In: FASE. LNCS, vol. 2306, pp. 174–188. Springer (2002)
22. Lluch Lafuente, A., Meseguer, J., Vandin, A.: State space c-reductions of concurrent systems in rewriting logic. In: Aoki, T., Taguchi, K. (eds.) Formal Methods and Software Engineering. pp. 430–446. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)
23. Lorenzo, C., Marco, G., Mauro, I.: Reconfigurable stochastic multi-formalism models: an approach based on maude. In: Proença, J., Umberto, R. (eds.) Software Engineering and Formal Methods. SEFM 2025 Collocated Workshops (to appear in). Springer Nature Switzerland, Cham (2026)
24. Meseguer, J.: Twenty years of rewriting logic. The Journal of Logic and Algebraic Programming **81**(7), 721–781 (2012). https://doi.org/https://doi.org/10.1016/j.jlap.2012.06.003, rewriting Logic and its Applications
25. Moscato, F., Flammini, F., Di Lorenzo, G., Vittorini, V., Marrone, S., Iacono, M.: The software architecture of the OsMoSys Multisolution Framework (2007)
26. Padberg, J., Kahloul, L.: Overview of reconfigurable petri nets. In: Heckel, R., Taentzer, G. (eds.) Graph Transformation, Specifications, and Nets: In Memory of Hartmut Ehrig, pp. 201–222. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-75396-6_11
27. Padberg, J., Schulz, A.: Model checking reconfigurable petri nets with maude. In: Echahed, R., Minas, M. (eds.) Graph Transformation. pp. 54–70. Springer (2016). https://doi.org/10.1007/978-3-319-40530-8_4
28. Reisig, W.: Petri Nets: An Introduction. Springer-Verlag New York, Inc., New York, NY, USA (1985). https://doi.org/10.1007/978-3-642-69968-9
29. Rubio, R., Martí-Oliet, N., Pita, I., Verdejo, A.: Qmaude: Quantitative specification and verification in rewriting logic. In: Chechik, M., Katoen, J.P., Leucker, M. (eds.) Formal Methods. pp. 240–259. Springer International Publishing, Cham (2023)
30. Sanders, W.: Integrated frameworks for multi-level and multi-formalism modeling. In: Petri Nets and Performance Models, 1999. Proc. 8th Int. Work. on. pp. 2–9 (1999)
31. Trivedi, K.S.: Sharpe 2002: Symbolic hierarchical automated reliability and performance evaluator. In: DSN '02: Proceedings of the 2002 International Conference on Dependable Systems and Networks. p. 544. IEEE, Washington, DC, USA (2002)
32. Ucar, I., Smeets, B., Azcorra, A.: simmer: Discrete-event simulation for r. Journal of Statistical Software **90**(2), 1–30 (2019). https://doi.org/10.18637/jss.v090.i02
33. Ölveczky, P.C., Meseguer, J.: Specification of real-time and hybrid systems in rewriting logic. Theoretical Computer Science **285**(2), 359–405 (2002). https://doi.org/https://doi.org/10.1016/S0304-3975(01)00363-2, rewriting Logic and its Applications